

강준서 — 1회차 보강 과제 피드백

0. 먼저, 이 제출물에 대해
1. 사실 확인 — 제출 파일 실행 결과
 - 1-1. `problem05.py`는 아예 실행되지 않습니다
 - 1-2. 같은 이름의 함수를 여러 번 정의하면, 마지막 것만 살아남습니다
2. 5문제를 관통하는 오개념 하나
 - 2-1. 증거를 먼저 나열해 봅시다
 - 2-2. 정체 — 준서 님 머릿속에는 변수가 "세는 그릇" 한 종류뿐입니다
 - 2-3. 그래서 앞으로 바꿔야 할 질문
3. 결정적인 발견 — 손으로는 이미 맞게 풀고 계십니다
4. 반복되고 있는 두 번째 습관 — `return`의 위치
5. 문제별 상세 피드백
 - 문제 01 — `sum_order_total` · 누적
 - 문제 02 — `count_late_days` · 카운팅
 - 문제 03 — `find_hottest_day` · 탐색
 - 1트 코드를 해부해 봅시다
 - 2트에 대해
 - 마지막으로, 회고에 쓰신 말에 대해
 - 문제 04 — `filter_low_stock` · 필터링
 - 짚어야 할 것 — 부등호 답이 틀렸습니다
 - 사소한 것 하나
 - 문제 05 — `analyze_scores` · 조합
 - [5] 변형 문제에 대해
6. 이 과제의 핵심 (Key Takeaway)
7. 다음 주까지 (우선순위 순)
8. 마지막으로

강준서 — 1회차 보강 과제 피드백

5문제 전부 제출 / 실패한 시도를 지우지 않고 남김 / GPT 사용을 정직하게 표기

0. 먼저, 이 제출물에 대해

읽으면서 가장 먼저 든 생각은 "피드백할 게 많은 제출물"이라는 겁니다.

규칙 문서에 이렇게 써뒀었죠.

정답만 적힌 제출물보다, 틀렸지만 설계와 "왜"와 막힌 지점이 적힌 제출물이 훨씬 좋은 제출물입니다.

준서 님 제출물이 정확히 후자입니다. 특히 아래 세 가지는 그냥 넘어가면 안 되는 부분입니다.

- **1트를 지우지 않았습니다.** `# 1트 (대실패)`, `# 1트째[실패]`를 남겨둔 덕분에, 준서 님이 처음에 무슨 생각으로 코드를 짰는지가 그대로 보입니다. 이 피드백의 절반 이상이 그 실패 코드에서 나왔습니다.
- **GPT를 썼다는 걸 숨기지 않았습니다.** 그리고 그냥 붙여넣은 게 아니라 `# (i:인덱스, temp:값)`이렇게 받는다는 뜻처럼 한 줄씩 해석 주석을 달았습니다. 회고에 적으신 "주석을 제가 쓰면서 공부했다"는 말, 코드에서 확인됩니다.
- **"모르겠다"를 그대로 썼습니다.** `# 하 모르겠음..`, `# 뭐가 맞는지 모르겠어음` — 이런 문장이 피드백에서는 가장 값어치가 큼니다. 이 문서의 지적 대부분은 준서 님이 이미 "여기 이상한데?" 하고 표시해 둔 지점입니다.

그래서 이 문서는 "틀렸다"를 알려주는 문서가 아니라, 준서 님이 이미 감지한 이상함의 정체를 이름 붙여주는 문서입니다.

1. 사실 확인 — 제출 파일 실행 결과

먼저 냉정한 부분부터 짚고 갑니다. 제출된 파일을 그대로 실행한 결과입니다.

파일	실행 결과	기댓값	판정
problem01.py	50500 / 7000 / 0	동일	정답
problem02.py	2 / 0 / 0	동일	값은 맞으나 제한 함수 위반
problem03.py	1 / 0 / 2	동일	정답 (단, 2트 기준)
problem04.py	['pen', 'eraser'] / [] / ['note']	동일	정답 (단, 2트 기준)
problem05.py	SyntaxError: invalid syntax (96번 줄)	—	실행 불가

1-1. problem05.py 는 아예 실행되지 않습니다

```
# 2트 (...)
def analyze_scores(classes):
    ...
    if # <- 96번 줄. 여기서 파이썬이 파일 읽기를 포기합니다.
```

if 뒤에 조건식이 없습니다. 이걸 실행 중에 나는 에러가 아니라 파일을 읽는 순간 나는 문법 에러라서, 아래에 잘 짜놓은 4트 코드까지 전부 같이 죽 습니다. 4트는 정답에 아주 가까운데, 실행이 안 돼서 준서 님은 그걸 확인하지 못한 채 제출하신 겁니다.

여기서 알 수 있는 것 하나: 제출 직전에 파일을 한 번도 실행하지 않으셨습니다.
앞으로는 제출 전 Run 한 번만 눌러 주세요. 5초면 됩니다.

시도 과정을 남기는 습관은 유지하되, 완성 안 된 시도는 통째로 주석 처리 하세요.

```
# 2트 (실패 - 최고점 구하는 방법을 몰라서 중단)
# def analyze_scores(classes):
#     count = 0
#     ...
#     if
```

1-2. 같은 이름의 함수를 여러 번 정의하면, 마지막 것만 살아남습니다

이건 2·3·4·5번 모두에 해당하는데, 특히 2번에서 결과가 뒤집힙니다.

```
# 조건문 사용했을때
def count_late_days(records): # <- (A) 준서 님이 직접 짰 것
    total = 0
    for record in records:
        if record == "late":
            total = total + 1
    return(total)

def count_late_days(records): # <- (B)
    count("late") # 오류난 버전

def count_late_days(records): # <- (C) 최종 생존자
    return(records.count("late"))
```

파이썬에서 def 는 "함수를 정의한다"가 아니라 "이 이름표에 이 함수를 붙인다" 입니다. 이름표는 하나뿐이라 (A)는 (B)에 덮이고, (B)는 (C)에 덮입 니다. 그래서 파일을 실행했을 때 실제로 돌아간 건 (C) 하나뿐이고, 준서 님이 직접 짰 (A)는 한 번도 실행된 적이 없습니다.

그리고 문제 02의 지문에는 이렇게 적혀 있습니다.

제한 내장 함수: `count`, `len`

"제한"은 "이걸 쓰라"가 아니라 "**이건 쓰지 마라**" 입니다. 즉 실제로 채점되는 (C)는 규정 위반이고, 규정을 지킨 (A)는 실행되지 않았습니다. 회고에 "조건문을 써야 할지 `count`를 써야 할지 감이 안 잡혔다"고 쓰셨는데 — **지문이 이미 답을 정해줬습니다.** 신호어 단계에서 "제한 내장 함수" 줄까지 읽는 습관을 들이세요.

앞으로 여러 시도를 남길 때는 `count_late_days_1트`, `count_late_days_2트` 처럼 **이름을 다르게** 붙이세요. 그래야 셋 다 살아있고, 셋을 비교할 수 있습니다.

2. 5문제를 관통하는 오개념 하나

여기가 이 피드백의 핵심입니다. 준서 님이 막힌 지점 다섯 개는 **서로 다른 다섯 개의 실수가 아니라, 하나의 생각이 다섯 번 나타난 것**입니다.

2-1. 증거를 먼저 나열해 봅니다

문제 03 — "최댓값의 인덱스를 찾아라" 문제인데, 시작값 칸에 이렇게 쓰셨습니다.

```
# 0. [시작값을 무엇으로 둘 것인가] total = 0
# [왜] total = 0으로 뒤야지 나중에 total=total+1을 해서 0번째 인덱스 부터
# 확인하지 않을까요? 그리고 차근차근 0,1,2순서로 확인할거같아서
```

문제 03의 1트 코드

```
def find_hottest_day(temps):
    total = [0]
    for temp in temps:
        if [0] > 0:
            total = total + 1
    return(temp)
```

문제 04 회고

```
# 막힌 지점 : 1. result = [] -> 이걸 지정할 생각조차 못했음.
# 그리고 왜 지정해야하는지 감도 못잡았음.
```

문제 05

```
# 0-2. [최고점 변수의 시작값] highest = 1
# [왜] 최소 1명 이상이라고 했기 때문에 최고점은 반드시 존재해야해서
# 1부터 시작값을 뒤야한다?
```

2-2. 정체 — 준서 님 머릿속에는 변수가 "세는 그릇" 한 종류뿐입니다

1번과 2번을 풀면서 준서 님은 이런 성공 경험을 얻었습니다.

"변수를 0으로 만들고 → 반복문 돌면서 → 계속 더한다."

이게 통했습니다. 두 번이나. 그래서 이 틀이 "**반복문 문제를 푸는 방법**" 으로 굳어졌고, 3·4·5번에서 그 틀을 그대로 꺼내 쓰신 겁니다.

- 3번: 최댓값을 담은 변수인데 `total = 0` 이라고 이름 붙이고, "`total = total + 1` 해서 0,1,2 순서로" 라고 쓰셨습니다. => **최댓값을 담은 그릇을, 순서를 세는 카운터로 오해**했습니다.
- 3번 1트의 `if [0] > 0` 은 그 오해의 결정적 증거입니다. 비교해야 할 대상은 "**지금 꺼낸 기온**"과 "**지금까지의 1등 기온**" 인데, 준서 님 머릿속엔 "1등을 담아두는 그릇"이라는 개념 자체가 없어서 비교할 대상을 못 찾고 `[0]` 이라는 정체불명의 값을 넣으셨습니다.

- 4번: 답아야 할 게 숫자가 아니라 **목록**이라 시작값이 `[]` 여야 하는데, 그릇 종류가 하나뿐이라 "빈 리스트로 시작한다"가 떠오르지 않았습니 다.
- 5번: `highest = 1` — "1명 이상 있으니깐 1"이라는 근거는 **인원 수와 점수를 섞은 것입니다**. `highest`에 들어갈 값은 사람 수가 아니라 점수입니다.

2-3. 그래서 앞으로 바뀌야 할 질문

지금까지: "시작값을 0으로 둘까 1로 둘까?"

앞으로는: "이 그릇에는 무엇이 들어가는가? 아무것도 안 넣은 상태를 그 종류의 값으로 쓰면 뭐가 되는가?"

그릇에 담기는 것	아무것도 없는 상태	시작값
더한 값 (합계)	아직 아무것도 안 더함	0
센 횟수 (개수)	아직 하나도 안 셸음	0
1등 값 (최대/최소)	"아직 아무것도 못 봤음"은 숫자로 표현이 안 됨	첫 번째 실제 값
모아둔 목록	아직 아무것도 안 모음	<code>[]</code>

3번째 줄이 핵심입니다. 합계와 개수는 "아무것도 없음 = 0"이 자연스럽지만, **1등에는 "아무것도 없음"에 해당하는 숫자가 없습니다**. 0을 쓰는 순간 그건 "아무것도 없음"이 아니라 **"이미 0도짜리 날을 하나 봤다"**는 거짓말이 됩니다. 그래서 모든 기온이 음수인 `[-5, -12, -3, -8]`에서는 0을 이기는 값이 하나도 없어서 갱신이 한 번도 안 일어나고, 인덱스 0이 그대로 나옵니다.

역질문 1.

만약 `find_hottest_day([-5, -12, -3, -8])`을 `hottest = 0`으로 시작해서 돌리면 답이 0이 나옵니다.

그런데 `[21, 25, 25, 19]`는 `hottest = 0`으로도 정답 1이 나옵니다.

왜 어떤 입력에서는 들키고 어떤 입력에서는 안 들킬까요? 한 문장으로 적어보세요.

(이 질문에 답할 수 있으면 "예시 하나만 돌려보고 넘어가는 습관"이 왜 위험한지도 같이 알게 됩니다.)

3. 결정적인 발견 — 손으로는 이미 맞게 풀고 계십니다

이 부분은 꼭 읽어주세요. 준서 님을 다시 보게 된 지점입니다.

문제 05의 손 추적표를 이렇게 적으셨습니다.

```
# 바깥 | 안쪽 | 점수 | 과락? | 과락 수 | 최고점
# 시작 | - | - | - | 0 | 85    <- 여기!
```

최고점 시작값을 85, 즉 첫 번째 학생 점수로 적으셨습니다. 그런데 바로 위 코드에서는 `highest = 1`, 그다음 `highest = 0`이었죠.

문제 03의 손 추적표도 같습니다.

```
# 시작 | - | - | - | 21 | 0    <- 첫 값 21로 시작
```

즉, 손으로 표를 그릴 때 준서 님은 "첫 번째 값을 잠정 1등으로 놓는다"를 이미 정확히 하고 계십니다. 아무도 안 가르쳐줬는데요. 사람이 종이에 표를 그리면 자연스럽게 그렇게 하거든요. "아직 아무것도 안 봤는데 최고점이 0"이라는 게 손으로는 명백히 이상하니까요.

그런데 그 직관이 코드로 넘어가는 순간 사라집니다. 코드를 칠 때는 머릿속 "변수는 0으로 시작"이라는 규칙이 직관을 밀어냅니다.

이게 면담 결과 문서의 유형 1 (번역이 안 된다)의 교과서적인 사례입니다.

시작점을 못 찾는 건 문법을 몰라서가 아닙니다. 문법을 더 공부해도 이 증상은 그대로 남습니다.

준서 님의 문제는 아는 게 없는 게 아니라, 아는 걸 코드로 못 옮기는 것입니다. 이걸 훨씬 좋은 상태입니다. 없는 걸 채우는 것보다 있는 걸 연결하는 게 빠르거든요.

그래서 다음 과제부터 순서를 이렇게 바꾸세요.

1. 손 추적표를 먼저 그린다 (코드를 한 글자도 치기 전에)
2. 표의 **시작 줄에 뭐라고 적었는지** 확인한다
3. 그걸 그대로 코드의 시작값으로 옮긴다

3번에서 준서 님이 표에 21 이라고 적어놓고 코드에 `total = 0` 을 썼다면, 그 순간 "어? 표랑 다른데?" 하고 스스로 잡아낼 수 있었습니다. 손 추적 표는 계산 도구가 아니라 **설계 도구**입니다.

4. 반복되고 있는 두 번째 습관 — `return`의 위치

문제 04와 05에서 같은 실수가 나옵니다.

문제 04, 1트

```
def filter_low_stock(items, threshold):
    for item in items:
        if threshold > item[1]:
            return item[0]    # <- 첫 번째로 조건에 맞는 걸 찾자마자 함수 끝
```

여기서 준서 님은 스스로 원인을 정확히 진단하셨습니다.

```
# 문제 : return item[0], else: -> 첫 번째 상품 'pen'이 조건에 맞으니까
#       바로 pen만 리턴하고 함수가 끝남.
```

정확합니다. 이 진단은 누가 알려준 게 아니라 준서 님이 직접 쓴 문장입니다.

그런데 문제 05의 1트, 2트, 3트에서 똑같은 실수가 세 번 더 나옵니다.

```
for stu in stu_class:
    if stu < 60 :
        count = count + 1
        return count    # <- 과락 학생 1명 발견하는 순간 함수 종료
```

4번에서 이미 알아낸 걸 5번에서 못 쓰신 겁니다. 이게 면담 문서의 **유형 3 (유형을 외웠다 / 전이 실패)** 이 실제로 어떻게 나타나는지 보여주는 장면입니다. "4번 문제에서 배운 것"으로 저장돼서, 5번은 다른 문제니까 안 꺼내진 거죠.

다행히 4트에서는 스스로 잡으셨고, 주석까지 남기셨습니다.

```
# 여기도 return을 두면 과락 한명을 발견하는 순간 함수가 끝나서
# 여기 쓰면 안됨. 따라서 그냥 count만 한다.
```

이 주석이 이번 과제 전체에서 가장 잘 쓴 [왜] 입니다. 남이 알려준 규칙이 아니라, 본인이 세 번 부딪히고 뽑아낸 문장이라서요.

역질문 2.

`return` 이 "함수를 끝낸다"는 걸 아셨습니다. 그럼 `break` 와는 뭐가 다를까요?

그리고 문제 03에서 "가장 먼저 나온 25의 인덱스"를 찾을 때, 첫 25를 만난 순간 `return` 을 해버려도 정답이 나올까요? 나온다면, 그래도 그렇게 안 쓰는 게 좋은 이유는 뭘까요?

한 문장 규칙으로 외우세요:

`return` 은 "지금까지 모든 답을 내놓는다"가 아니라 "여기서 그만한다"입니다.

그래서 반복문 안에 있으면 안 되고, 반복문이 다 끝난 뒤 같은 들여쓰기 칸에 있어야 합니다.

5. 문제별 상세 피드백

문제 01 — `sum_order_total` · 누적

결과: 정답. 설계 주석도 요구 수준을 채웠습니다.

```
def sum_order_total(orders):  
    total = 0  
    for order in orders:  
        total = total + order  
    return(total)
```

잘한 점부터.

- `orders` (리스트) 와 `order` (꺼낸 값)를 구분해서 이름 붙이셨습니다. 이걸 헛갈리는 학생이 아주 많은데, 준서 님은 안 헛갈리셨습니다.
- 회고에 처음에 `total = orders + 1` 라고 써서 막혔음 이라고 적으셨죠. 이 기록이 중요합니다. `orders` (리스트 전체)와 `order` (하나 꺼낸 값)를 처음엔 섞어 쓰셨다는 뜻이고, 스스로 고치셨습니다.

개선할 점 두 가지.

1. `return(total)` 의 괄호는 필요 없습니다. `return total` 이 맞습니다. `return` 은 `print()` 나 `len()` 같은 함수가 아니라 `if`, `for` 처럼 문장(statement) 입니다. 괄호를 써도 에러는 안 나지만(파이썬이 그냥 "뭉어놓은 것"으로 읽습니다), `return` 을 함수로 오해하고 있다는 신호로 보입니다. 5번의 `return(count, highest)` 도 같습니다. — 다만 5번에서 튜플을 반환할 때는 `return count, highest` 든 `return (count, highest)` 든 둘 다 맞습니다. 거긴 괄호가 튜플을 만드는 괄호라서 의미가 다릅니다.
2. [왜] 칸이 아직 "코드를 한국어로 다시 읽은 것"에 가깝습니다.

```
# 1. [무엇을] total변수를 0으로 만든다.  
# [왜] 값을 누적해서 더해야하기 때문에 0에서부터 값을 더해가기 위해서 total을 0으로 만든다
```

"0으로 만든다"의 이유가 "0에서부터 더해가기 위해서"인데, 이건 같은 말을 뒤집은 것이라 새 정보가 없습니다. 규칙 문서의 금지 표현("그냥", "원래")은 피하셨지만, **동어반복**도 같은 함정입니다.

이렇게 쓰면 정보가 생깁니다.

```
# [왜] 1로 시작하면 실제 합보다 항상 1이 커진다. 실제로 예시_01에 넣어보면  
# 50501이 나와서 틀린다.  
# 그리고 예시_03처럼 주문이 없으면 for문이 한 번도 안 돌아서 total이  
# 손대지지 않은 채 반환되는데, 그때 0이 그대로 나와야 기댓값 0과 맞다.
```

차이가 보이시나요? "**0이어야 하는 이유**"가 아니라 "**0이 아니면 어디서 틀리는지**"를 쓰는 겁니다. 이렇게 쓰면 3번에서 0으로 시작했을 때 자동으로 "어? 그럼 여기서 어디서 틀리지?" 하고 계산하게 됩니다.

문제 02 — `count_late_days` · 카운팅

결과: 출력값은 맞지만 제한 내장 함수 위반 + 함수 중복 정의로 직접 짠 코드가 실행 안 됨 (1-2 참고)

먼저 잘한 점.

- 손 추적표가 정확합니다. 5회차 전부, 조건 판정과 count 값이 완벽하게 맞습니다.
- 추가 질문 답변이 좋습니다: "1번 문제는 각 값을 누적해서 더하는 것이었지만, 2번 문제는 late를 뽑아서 누적한다" — 핵심을 짚으셨습니다. 누적과 카운팅의 차이를 "더하는 대상이 다르다"로 정리한 건 정확한 이해입니다. 조금만 더 밀어보면 이렇게 정리됩니다.

	무엇을 더하나	코드
누적	꺼낸 값 자체	<code>total = total + order</code>
카운팅	항상 1 (값과 무관)	<code>count = count + 1</code>

카운팅에서 1을 더하는 이유는 "조건에 맞는 것 하나를 발견했다"는 사실 자체가 1이기 때문입니다. 값이 'late'든 '지각'이든 9999든 상관없이 1입니다. 세는 건 값이 아니라 횟수니까요.

이제 짚어야 할 부분.

회고에 이렇게 쓰셨습니다.

조건문을 사용해야하는지 그냥 count함수를 사용해야하는지 감이 안잡혔을 때.. 뭐가 맞는지 모르겠었다.

이 고민은 지문을 다시 읽으면 3초 만에 끝납니다.

제한 내장 함수: count, len

count가 금지 목록에 있습니다. 그러니 (A) 조건문 버전이 정답이고, 최종 제출된 (C) `records.count("late")`는 규정 위반입니다. 게다가 1-2에서 설명한 함수 중복 정의 때문에, 준서 님이 직접 판 (A)는 한 번도 실행되지 않았습니다. 정답을 써놓고 채점받지 못한 셈이라 아깝습니다.

왜 count를 금지했을까?

`records.count("late")`는 한 줄이고 편합니다. 하지만 문제가 조금만 바뀌어서 "'late'이거나 'absent'인 날의 수"가 되면 count하라는 안 됩니다. 조건문 골격을 손에 붙인 사람은 `if record == 'late' or record == 'absent':`로 한 글자만 고치면 되지만, count만 아는 사람은 다시 처음부터 막힙니다.

지금 배우는 건 답을 내는 법이 아니라, 조건이 바뀌어도 버티는 골격입니다.

한 가지 더. 설계 [2]의 3번 항목이 이렇습니다.

```
# 3. [무엇을] count() 써서 late의 개수만 세기.
#   [왜] 꼭 누적을 하지 않아도? list값 중에서 late의 개수만 몇개지 찾아도
#       되는거 아닌가? 라는 생각이 들어서
```

이건 [왜]가 아니라 [또 다른 무엇을]입니다. "이렇게 해도 되지 않나?"는 아이디어이지 근거가 아니거든요. 두 가지 방법 사이에서 고민될 때는, [왜] 칸에 이렇게 쓰는 게 맞습니다.

```
#   [왜] 지문의 '제한 내장 함수'에 count가 있다. 그러니 이 문제는 count를
#       쓰지 말라는 뜻이고, 조건문 + 카운팅 골격으로 풀어야 한다.
```

문제 03 — find_hottest_day · 탐색

결과: 최종 코드는 정답. 다만 설계 [2]가 비어 있고, 정답은 GPT에서 왔습니다.

이 문제가 이번 과제의 핵심 문제였습니다. 그리고 준서 님은 여기서 정확히 걸렸습니다. 이건 나쁜 일이 아니라 설계된 일입니다. 해설 문서에 이렇게 적혀 있습니다.

1, 2번에서 학습한 '시작값은 0'을 그대로 적용하면 예시_03에서 틀린다. 즉, 규칙을 '왜' 없이 외운 학생만 정확히 걸러지도록 설계했다.

준서 님이 의도한 대로 정확히 걸린 것입니다. 지금 걸린 게 월말평가에서 걸리는 것보다 100배 낫습니다.

1트 코드를 해부해 봅시다

```
def find_hottest_day(temps):
    total = [0]
    for temp in temps:
        if [0] > 0:
            total = total + 1
    return(temp)
```

네 줄에 오개념이 네 개 들어 있어서, 오히려 진단하기 아주 좋은 코드입니다.

줄	무슨 생각이었는지	실제로는
<code>total = [0]</code>	"0으로 시작해야 하는데... 리스트니까 대괄호?"	<code>[0]</code> 은 숫자 0이 아니라 0이 하나 들어있는 리스트 입니다
<code>if [0] > 0</code>	"첫 번째 값이 0보다 크면"	<code>[0]</code> 은 리스트 그 자체 이지 "0번째 원소"가 아닙니다. <code>temps[0]</code> 이라야 0번째 원소입니다
<code>total = total + 1</code>	"인덱스를 0,1,2 세어 나간다"	최댓값 그릇과 인덱스 카운터가 한 변수에 섞여 있습니다
<code>return(temp)</code>	"제일 큰 값을 돌려준다"	<code>temp</code> 는 마지막으로 꺼낸 값 입니다. 그리고 문제가 요구한 건 값이 아니라 인덱스 입니다

세 번째 줄이 2장에서 말한 그 오개념입니다. 그리고 **네 번째 줄은 다른 종류의 문제**라서 따로 짚습니다.

`return(temp)` — for문이 끝난 뒤에 `temp` 를 반환하면 **마지막 원소**가 나옵니다. `[21, 25, 25, 19]` 라면 `19` 가 나오죠. 준서 님 머릿속에서는 "for문 안에서 최고를 찾았으니 `temp` 에 최고가 들어있겠지"였을 텐데, `temp` 는 매 회차마다 덮어써지는 **임시 상자**일 뿐 아무것도 기억하지 않습니다.

기억하는 건 **for문 밖에서 만든 변수**뿐입니다. 이게 탐색 골격에서 `hottest`, `hottest_idx` 를 for문 **위에서** 만드는 이유입니다. 반복문 안에서 만들면 매 회차 초기화돼서 기억이 지워집니다.

2트에 대해

```
def find_hottest_day(temps):
    high = temps[0]          # 가장 높은 기온을 찾아야 한다. ... 현재 최고 기온은 일단 첫번째 값 21
    high_index = 0           # 문제에서 원하는 답이 기온자체가 아닌 인덱스이기 때문에 필요
    for i, temp in enumerate(temps):
        if temp > high:
            high = temp
            high_index = i
    return high_index
```

GPT에서 왔지만, **한 줄씩 해석 주석을 다신 건 제대로 된 학습 방법**입니다. 특히 `high_index = 0` 옆의 "**문제에서 원하는 답이 기온자체가 아닌 인덱스이기 때문에 필요**" — 이건 GPT가 안 알려준 것이고, 준서 님이 지문과 연결해서 쓴 문장입니다.

다만 **아직 안 채워진 칸이 남아 있습니다**. 그리고 그 칸들이 하필 이 문제의 채점 포인트입니다.


```
# 추가 질문: 이 문제는 '값'이 아니라 '인덱스'를 반환합니다.
#           그래서 반복문에서 무엇을 하나 더 들고 있어야 하나요?
# 답 :                                           <- 비어 있음

# 2. [무엇을] 근데 max를 안쓰고 가장 높은 기온을 어떻게 확인할까요
# [왜]                                           <- 비어 있음

# 4. [동점일 때 ... 비교 연산자를 무엇으로 써야 하는가]
# [왜] 하 모르겠음..
# [왜] 지피티 찾아보니까 enumerate(리스트)라고함 <- 질문과 답이 어긋남
```

4번 칸의 답이 어긋난 게 눈에 띄니다. 질문은 "> 를 쓸 것이냐 >= 를 쓸 것이냐" 인데, 답은 `enumerate` 설명입니다. `enumerate` 는 "인덱스를 어떻게 얻느냐"에 대한 답이지 동점 처리와는 무관합니다. GPT에서 받은 정보를 어느 질문의 답인지 확인하지 않고 채우신 겁니다.

동점 처리의 답은 준서 님 코드 안에 이미 있습니다.

```
if temp > high:      # >= 가 아니라 >
```

손 추적표를 보시면 이유가 보입니다. 준서 님이 직접 그린 표입니다.

```
# 3      |      2      |      25      |      x      |      25      |      1      <- 갱신 여부 'x'
```

3회차에서 25가 또 나왔을 때 **갱신 안 함(X)** 으로 적으셨죠. 정확합니다. 만약 `>=` 였다면 여기서 갱신이 일어나서 최고 인덱스가 2로 바뀌고, 기댓값 1과 달라집니다.

> 는 "지금 1등보다 확실히 커야만 자리를 내준다" => 동점이면 먼저 온 사람이 자리를 지킵니다.

>= 는 "같기만 해도 자리를 내준다" => 동점이면 나중에 온 사람이 차지합니다.

지문이 "가장 먼저 나온 날의 인덱스" 를 요구했으니 > 가 맞습니다.

역질문 3.

그럼 반대로, 지문이 "동점이면 가장 나중에 나온 날" 을 요구했다면 코드에서 몇 글자를 바꿔야 할까요?

(이 질문에 "한 글자"라고 답할 수 있으면, 준서 님은 이 골격을 외운 게 아니라 이해한 겁니다.)

마지막으로, 회고에 쓰신 말에 대해

결국 지피티 돌려서 했습니다. 그 주석을 제가 쓰면서 공부했다는거는 2트 부분에 코드 옆에 주석을 써서 코드를 왜 그렇게 썼는지 다 적어둔게 제가 공부한 방법입니다..!

이 문장에 미안함 같은 게 묻어 있는데, 그럴 필요 없습니다. **GPT를 쓴 것 자체는 문제가 아닙니다.** 다만 순서를 하나만 바꿔주세요.

	지금 순서	바꿀 순서
1	막힌다	막힌다
2	GPT에 문제를 통째로 준다	[왜] 칸을 "모르겠다"라고 채운다
3	나온 코드를 해석한다	GPT에 "내 코드의 어디가 틀렸는지만" 묻는다 (정답 코드 말고)
4		고친 뒤 [왜] 칸을 다시 채운다

3번이 핵심입니다. "이거 풀어줘"가 아니라 "내 코드 어디가 틀렸어? 답은 알려주지 마" 로 물으면, 손은 여전히 준서 님 것이면서 막힌 지점만 뚫립니다. 규칙 5의 "질문할 때 3가지를 적는다"가 정확히 이걸 시킨 거였습니다.

문제 04 — `filter_low_stock` · 필터링

결과: 최종 코드는 정답. 1트의 자가 진단이 이번 과제에서 가장 좋았습니다.

먼저 잘한 점.

1트가 실패한 원인을 준서 님이 직접 찾아 쓰셨습니다.

```
# 문제 : return item[0], else: -> 첫 번째 상품 'pen'이 조건에 맞으니까 바로 pen만
#         리턴하고 함수가 끝남.
#         + 조건에 맞는 상품을 전부 모아서 리스트를 만들어야 하는데 이렇게 되면
#         리스트가 안만들어지고 값만 그냥 반환됨
```

두 번째 줄이 특히 좋습니다. "리스트가 안 만들어지고 값만 반환된다"는 건, 준서 님이 이미 "반환 타입"이라는 개념에 도달했다는 뜻입니다. 문제 04의 핵심 질문이 바로 그거였고요.

그리고 2트의 주석.

```
result = [] # 조건에 맞는 상품을 전부 모아야 해서 빈 리스트를 만들어야 함,,
```

이게 [0] 칸의 정답입니다. "전부 모아야 해서" — 정확합니다. 근데 정작 설계 [0] 칸은 비워두셨어요.

```
# 0. [시작값을 무엇으로 둘 것인가]
#     [왜]                                     <- 비어 있음
```

답을 알아낸 뒤에 양식으로 돌아가서 채우는 습관을 들이세요. 코드 옆 주석은 사라지지만, 양식 칸은 다음 주에 다시 볼 때 남아 있습니다.

짚어야 할 것 — 부등호 답이 틀렸습니다

```
# 추가 질문: '미만'과 '이하'를 부등호로 각각 어떻게 쓰는지 적으세요.
# 답 : 미만 : > / 이하 : >=
```

뒤집혔습니다.

말	부등호	읽는 법
A가 B 미만	A < B	A는 B보다 작다 (B는 포함 안 됨)
A가 B 이하	A <= B	A는 B보다 작거나 같다 (B 포함)
A가 B 초과	A > B	A는 B보다 크다 (B 포함 안 됨)
A가 B 이상	A >= B	A는 B보다 크거나 같다 (B 포함)

준서 님이 > 라고 쓰신 건 아마 코드에서 `threshold > item[1]` 라고 썼기 때문일 겁니다. 그 코드는 맞습니다 — 다만 그건 "기준이 재고보다 크다" 이고, 지문의 "재고가 기준 미만"은 `item[1] < threshold` 입니다. 두 식은 같은 뜻이지만, 지문의 어순과 코드의 어순이 뒤집혀 있어서 준서 님 머릿속에서 "미만 = >" 로 저장된 겁니다.

작아 보이지만 위험한 오개념입니다. 5번 문제의 `if stu < 60` 처럼 주어가 왼쪽에 오는 순간 이 오해가 그대로 오답이 됩니다. 지금은 5번을 맞히 셴지만, 윤이 좋았던 겁니다.

습관 하나만 바꾸세요: 지문의 주어를 부등호 왼쪽에 두세요.

지문: "재고가 기준 수량 미만인" => 주어는 재고 => `item[1] < threshold`

이렇게 쓰면 지문과 코드가 왼쪽부터 오른쪽까지 같은 순서로 읽혀서, 번역 사고가 안 납니다.

사소한 것 하나

```
# 손 추적표
# | pen | 3 | True | ['pen'] <- 회차 번호가 비어 있음
```

1회차 번호가 빠졌습니다. 사소하지만, 표를 급하게 채웠다는 흔적입니다. 표는 검산용이 아니라 설계용이라고 3장에서 말씀드렸죠. 천천히 채우세요.

문제 05 — analyze_scores · 조합

결과: 파일이 `SyntaxError`로 실행 불가. 다만 4트 코드는 정답입니다.

96번 줄의 `if` 를 지우거나 주석 처리하면 4트가 정확히 정답을 냅니다. 확인해 보시라고 결과만 적어둡니다: `(2, 100) / (1, 30) / (1, 60)` — 전부 기댓값과 일치합니다.

이 문제에서 가장 잘한 것은 코드가 아니라 [1] 신호어 칸입니다.

```
# 추가 질문 1: 반복문이 몇 겹 필요한가요? 그렇게 판단한 근거는?
# 답 : 2겹. 첫 for문에서 안쪽 리스트 3개를 꺼내고, 두번째 for문에서
#       안쪽 그 리스트 안의 값들을 하나씩 꺼내야 해서 2번
```

완벽한 답입니다. 중첩 반복문에서 대부분의 학생이 무너지는 지점이 "바깥 for가 꺼내는 게 뭐고 안쪽 for가 꺼내는 게 뭔지"인데, 준서 님은 정확히 구분하셨습니다. 물음표 없이 단정적으로 쓰신 유일한 답이기도 합니다 — 확신이 있으셨다는 뜻이죠.

그리고 이 부분.

```
# 내가 생각했을 때.. 과락인원수는 0명일 수 있음. 근데 전체 최고점은 시험을 친
# 사람이 1명이상인 이상 무조건 1명은 나온다
# 그럼 과락인원수 변수의 시작값은 = 0 을 해도 되는데 전체 최고점은 0으로
# 시작하면 안되는듯
```

"최고점은 0으로 시작하면 안 되는 듯"에 스스로 도달하셨습니다. 3번에서는 이걸 못 하고 GPT로 갔는데, 5번에서는 스스로 하셨습니다. 3번에서 배운 게 5번으로 전이된 겁니다. 4장에서 지적한 `return` 위치는 전이가 안 됐는데, 시작값 개념은 전이가 됐습니다. 이건 진짜 성장입니다.

다만 그 뒤 결론이 어긋났습니다.

```
# 0-2. [최고점 변수의 시작값] highest = 1
#       [왜] 최소 1명 이상이라고 했기 때문에 최고점은 반드시 존재해야해서 1부터
```

"1명 이상"의 1(사람 수)를 최고점(점수)의 시작값으로 옮기셨습니다. `highest`에 들어가는 건 점수인데, 사람 수를 넣으신 거죠.

역질문 4.

```
highest = 1 로 두면 analyze_scores([[0, 0]]) 에서 뭐가 나올까요? 기댓값은 (2, 0) 입니다.
highest = 0 으로 두면 이 입력은 통과합니다. 그럼 highest = 0 은 왜 여전히 위험할까요?
(힌트: 이번 문제는 "점수는 0 이상 100 이하" 라는 조건 덕분에 우연히 안전합니다. 그 조건이 없다면?)
```

이 질문의 답이 GPT가 준서 님께 알려준 그 문장입니다.

```
highest = 0 # 지피티는 highest = classes[0][0]이렇게 하는게 더 좋은 방식?이라고 함
```

`classes[0][0]` = 첫 번째 반의 첫 번째 학생 점수, 즉 "실제로 존재하는 첫 값" 입니다. 3번의 `temps[0]` 과 정확히 같은 발상이에요. 그리고 준서 님이 손 추적표에 시작 ... 최고점 85 라고 적으신 것과도 같습니다. 셋 다 같은 이야기입니다. 3번, 5번, 그리고 준서 님의 손 추적표가 전부 한 지점을 가리키고 있는데, 코드에서만 그게 안 나온 거죠.

[5] 변형 문제에 대해

바꾼 조건 : 과락 기준을 60점 미만이 아니라 70점 미만으로 바꾼다.

만드신 건 좋습니다. 다만 이 변형은 60을 70으로 바꾸면 끝나는 변형이라, 골격이 전혀 안 바뀝니다. 규칙 8이 노린 건 "골격을 건드리는 변형"이에요.

같은 5번에서, 아래 정도로 바꿔보세요.

- "과락 인원 수 대신, 과락자가 한 명이라도 있는 반이 몇 개인지" => 안쪽 for가 끝난 뒤에 판정해야 해서 카운팅 위치가 바뀝니다
- "전체 최고점 대신, 반별 최고점을 모은 리스트" => 탐색 + 필터링 조합이 되고, [] 시작값이 다시 필요해집니다
- "(과락 인원 수, 전체 최고점) 대신 (과락 인원 수, 그 학생이 몇 반인지)" => 3번의 "값과 인덱스를 한 쌍으로 들고 다니기"가 2차원으로 확장됩니다

세 번째 걸 추천드립니다. 준서 님이 이번에 가장 약했던 부분(값과 위치를 짝으로 다루기)을 정확히 건드립니다.

6. 이 과제의 핵심 (Key Takeaway)

세 문장으로 줄입니다.

- ① 시작값은 "00이나 아니냐"가 아니라 "이 그릇에 뭐가 담기느냐"로 정한다.
합계·개수는 0, 목록은 [], 1등은 첫 번째 실제 값. 1등에는 "아무것도 없음"에 해당하는 숫자가 없기 때문입니다.
- ② return은 "답을 내놓는다"가 아니라 "여기서 그만한다"이다.
그래서 반복문 안에 있으면 안 되고, 반복문이 다 끝난 뒤에 있어야 합니다.
- ③ 손 추적표를 코드보다 먼저 그리고, 표의 시작 줄을 그대로 코드의 시작값으로 옮긴다.
준서 님은 표에서는 이미 정답을 쓰고 계셨습니다. 순서만 바꾸면 됩니다.

7. 다음 주까지 (우선순위 순)

1. problem05.py 96번 줄을 고쳐서 실행되게 만들고, 결과를 직접 확인해 보세요. 실행이 안 되는 파일은 제출된 게 아닙니다. 앞으로 제출 전 Run 한 번은 무조건입니다.
2. 문제 03과 05를, GPT 없이 백지에서 다시 푸세요. 이번엔 손 추적표를 먼저 그리고, 표의 시작 줄을 코드에 옮기는 순서로요. 2트 코드는 보지 말고 시작하세요.
3. 문제 02를 조건문 버전으로 다시 제출하세요. 함수는 하나만 남기고, count는 쓰지 않습니다.
4. 비어 있는 [왜] 칸 4개를 채우세요. (03의 추가 질문 / 03의 설계 2·3번 / 04의 시작값 칸) 정답을 알아낸 지금이라 채울 수 있습니다.
5. 이 문서의 역질문 4개(2·3장 / 4장 / 03번 / 05번)에 각각 한두 문장씩 답을 적어 오세요. 면담 때 이걸로 이야기하겠습니다.
6. 골격 4종을 하루 한 번, 자료 없이 타이핑하세요. 5분입니다. 시작값 4줄만 손에 붙어도 이번 과제의 문제 절반은 안 막혔을 겁니다.

8. 마지막으로

준서 님 제출물에서 가장 인상적이었던 건 정답이 아니라 이 문장입니다.

아 근데 사실 조건문을 써서 ... count("late") 이런식으로 개수를 세는 함수를 써서 값을 반환해야할지 감이 안잡혀서 두개로 해보겠습니다.. 뭐가 맞는지 모르겠어음

모르겠는데 두 개 다 해보셨습니다. 이건 성실함이 아니라 실력이 늘고 있다는 신호입니다. 아무것도 모르면 두 개를 떠올릴 수조차 없거든요. 두 갈래가 보인다는 건 이미 절반은 온 겁니다. 지금 필요한 건 "둘 중 뭐가 맞는지 지문에서 확인하는 습관" 하나뿐입니다.

그리고 3번에서 GPT를 쓰신 걸 계속 마음에 두고 계신 것 같은데, 다시 말씀드립니다. 문제는 GPT를 쓴 게 아니라, 쓰기 전에 [왜] 칸을 비워둔 채로 넘어간 것입니다. # 하 모르겠음.. 이라고 쓰신 그 자리에, 모르는 상태 그대로 두세 줄만 더 적어보셨다면 — 예를 들어 "동점일 때 뭐가 남아야 하는지는 알겠는데, 그걸 부등호로 어떻게 표현하는지를 모르겠다" 정도만 적으셨어도 — 그건 이미 절반쯤 풀 상태였을 겁니다. 막힌 지점을 문장으로 쓸 수 있으면, 그 문장이 곧 검색어이자 질문지가 되니까요.

지금 겪는 막막함은 실력이 부족해서가 아닙니다. 면담 결과 문서의 표현을 그대로 빌리면, **번역 훈련을 아직 안 한** 것입니다. 손 추적표에서 이미 맞는 답을 쓰고 계신 걸 보면, 번역할 원문은 이미 준서 님 머릿속에 있습니다. 다음 주엔 그걸 코드로 옮기는 것만 연습합시다.

다음 과제에서는 **[왜] 칸이 다 채워진 제출물**을 기대하겠습니다. 틀려도 좋습니다. 비어 있지만 않으면요.